

END-TERM PROJECT - 01 REPORT

ON

CAR DODGING GAME

B.Tech. Computer Science and Engineering

Semester: 6th

Section: 23412C1

Course Title & Code: Game Programming With C++ (CSE 3545)

Submitted By:

Student Name: Swayam Swarup Jethi

Registration No: 2341010085

Guided By: Prof. Mohammad Maksood Akhter

Academic Year 2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF
TECHNICAL EDUCATION AND RESEARCH
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY
BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project focuses on the design and development of a simple 2D Car Dodging Game using C++ and SFML (Simple and Fast Multimedia Library). The game allows the player to control a car and avoid collisions with incoming enemy vehicles moving across different lanes. Developed using Object-Oriented Programming concepts, the project includes features such as player movement, random enemy car generation, increasing game speed, collision detection, score calculation, pause/resume functionality, restart options, and sound effects. SFML graphics and audio modules are used to create an interactive graphical interface and smooth gameplay experience. The project provides practical understanding of real-time rendering, event handling, animation, inheritance, and multimedia programming, while demonstrating the implementation of interactive 2D game development using modern C++ techniques.

Submitted By

Guided By

TABLE OF CONTENTS

Section No.	Topic	Page No.
01	Introduction	1
02	Problem Statement	2
03	Objectives of the Project	3
04	Tools and Technologies Used	4
05	Methodology	5
06	Results Analysis and Screenshots	6-7
07	Logical Understanding	8
08	Conclusion	9
09	Future Scopes and Application	10
	References	11
	Annexure	12-18

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Main Menu Screen	6
Figure 2	Gameplay Screen	6
Figure 3	Pause Screen	7
Figure 4	Game Over Screen	7

01. Introduction

The Car Dodging Game is a simple 2D arcade-style driving game developed using C++ and SFML (Simple and Fast Multimedia Library). In this game, the player controls a car and avoids collisions with enemy vehicles moving along different lanes. The project is designed to demonstrate the basic concepts of game development and multimedia programming using modern C++ techniques.

The game includes features such as player movement, random enemy car spawning, increasing game speed, collision detection, score tracking, pause/resume functionality, restart options, background music, and sound effects. The project is developed using Object-Oriented Programming concepts with separate classes such as Car, PlayerCar, and EnemyCar to improve modularity and code reusability. SFML graphics and audio libraries are used to provide smooth gameplay and an interactive graphical user experience.

02. Problem Statement

The problem addressed in this project is to design and implement a simple 2D Car Dodging Game using C++ and SFML.

The game is developed to demonstrate real-time game mechanics where the player controls a car and avoids collisions with enemy vehicles moving along different lanes.

The project uses concepts such as object-oriented programming, event handling, animation, graphics rendering, sound integration, and collision detection to create an interactive gaming application.

03. Objectives of the Project

The objectives of this project are defined to provide a clear direction for the design and development of the Car Dodging Game. The objectives of this practical project are:

- To develop a simple 2D car dodging game using C++ and SFML.
- To implement Object-Oriented Programming concepts using classes such as Car, PlayerCar, and EnemyCar.
- To implement player car movement using keyboard controls.
- To generate enemy vehicles randomly in different lanes.
- To implement collision detection between player and enemy vehicles.
- To create a real-time score and speed management system.
- To gradually increase game difficulty through speed and traffic density.
- To integrate graphics, textures, sound effects, and background music using SFML.
- To implement pause, resume, and restart functionalities.
- To enhance practical understanding of graphics programming, event handling, and game development using C++.

04. Tools and Technologies Used

The following tools and technologies were used in this project:

Programming Language: C++

Graphics and Audio Library: SFML (Simple and Fast Multimedia Library)

Compiler: G++

Development Environment: Visual Studio Code

Operating System: Ubuntu WSL (Windows Subsystem for Linux)

These tools provide a suitable environment for writing, compiling, and executing C++ graphical game applications.

05. Methodology

The methodology followed during the development of this project includes:

- **Understanding the Problem:** Identifying the game requirements, lane system, enemy car movement, collision detection, and expected gameplay behavior.
- **Game Design:** Designing the road layout, player movement system, enemy spawning mechanism, scoring system, and speed progression.
- **Program Development:** Writing the game logic and graphical implementation using C++ and SFML.
- **Object-Oriented Implementation:** Creating separate classes such as Car, PlayerCar, and EnemyCar using inheritance and modular programming concepts.
- **Compilation and Execution:** Compiling and executing the program using the G++ compiler in Ubuntu WSL.
- **Testing and Debugging:** Checking for errors related to movement, collisions, rendering, and audio playback, and improving game performance.
- **Final Output:** Ensuring proper gameplay mechanics, increasing difficulty, score tracking, sound effects, and smooth execution of the game.

06. Results Analysis and Screenshots

This section describes the results obtained after implementing the Flappy Bird Game using C++ and SFML. The game was successfully executed and produced the expected graphical and interactive output.

The implementation demonstrates the correct use of object-oriented programming concepts, collision detection, real-time rendering, event handling, and multimedia integration.

Output Screenshots:

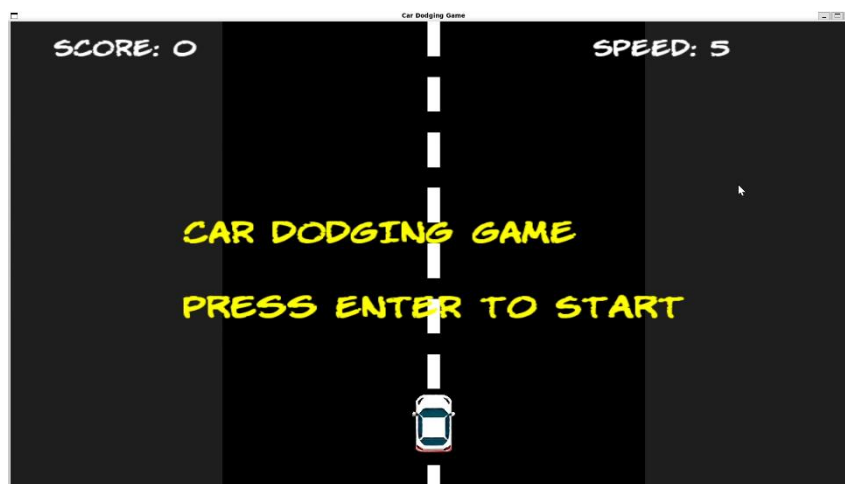


Figure 1 : Main Menu Screen

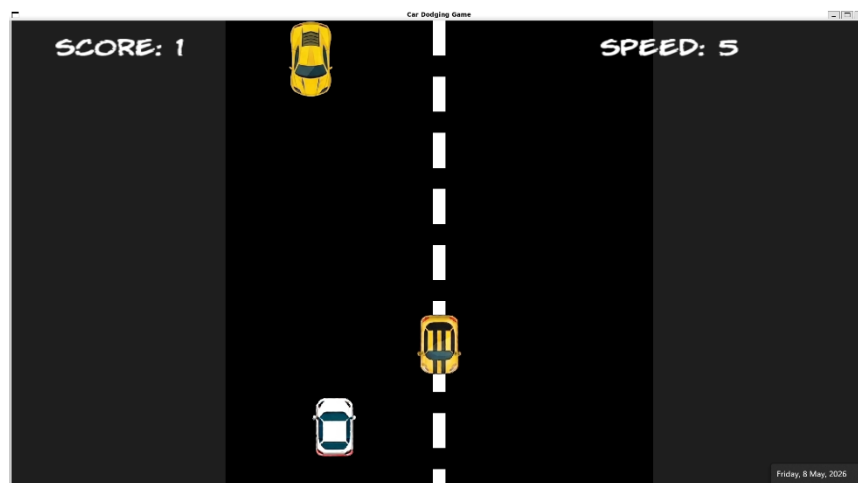


Figure 2 : Gameplay Screen

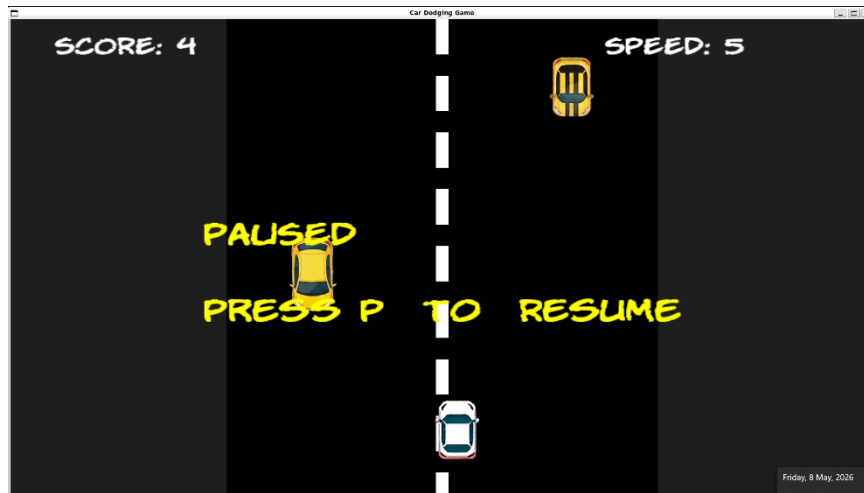


Figure 3 : Pause Screen

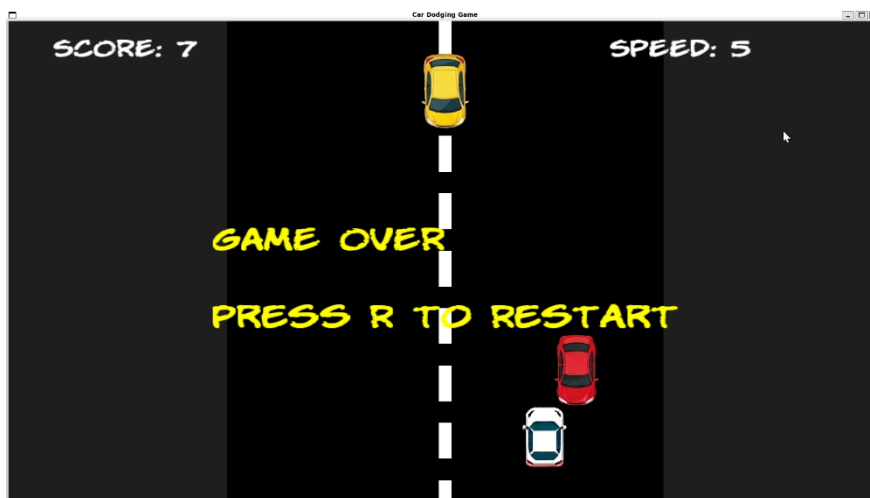


Figure 4 : Game Over Screen

07. Logical Understanding

The logic of the program is based on:

- Handling keyboard input from the player for controlling the movement of the car across different lanes.
- Processing enemy car spawning, movement, collision detection, speed increase, and score calculation using conditions and loops.
- Applying Object-Oriented Programming concepts using classes such as Car, PlayerCar, and EnemyCar for modular and reusable coding.
- Using a real-time game loop for updating object positions, checking collisions, rendering graphics, and managing audio playback.
- Displaying the road, player vehicle, enemy vehicles, score, speed level, and game status clearly using SFML graphical components.

This project helps in understanding real-time game execution, event-driven programming, object-oriented design, multimedia integration, and logical flow control in C++ programming.

08. Conclusion

In conclusion, this project was successfully developed using the C++ programming language and SFML library. The project helped in understanding concepts such as object-oriented programming, collision detection, event handling, animation, and multimedia programming through the development of an interactive car dodging game.

The implementation of player controls, enemy car spawning, score tracking, increasing game speed, and audio integration improved practical programming and problem-solving skills. Overall, the project provided valuable experience in game development using C++ and SFML.

09. Future Scopes and Application

The future improvements and applications of this project include:

- Adding advanced features such as multiple vehicles, traffic signals, power-ups, fuel systems, and weather effects.
- Improving game efficiency and performance using advanced graphics programming and optimized game logic.
- Extending the game with additional levels, multiplayer functionality, adaptive AI traffic systems, and mobile platform support.
- Designing a more user-friendly interface with enhanced graphics, animations, sound systems, and menu screens.
- Implementing advanced file handling for storing player profiles, high scores, difficulty settings, and game progress.

Game development using C++ and SFML has applications in arcade game development, multimedia software systems, educational simulations, and interactive entertainment applications.

References

- (1) SFML Official Documentation – <https://www.sfml-dev.org/documentation/>
- (2) C++ Reference – <https://cplusplus.com/>
- (3) SFML Graphics Module Tutorial – <https://www.sfml-dev.org/tutorials/>
- (4) Visual Studio Code Documentation – <https://code.visualstudio.com/docs>
- (5) GNU G++ Compiler Documentation – <https://gcc.gnu.org/>

Annexure

The following section includes the code of the game including the explanation for different portions.

main.cpp

```
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <iostream>
#include <vector>
#include <ctime>
#include "PlayerCar.h"
#include "EnemyCar.h"

using namespace sf;
using namespace std;

const int WIDTH = 800, HEIGHT = 700;

enum GameState
{
    MENU,
    PLAYING,
    PAUSED,
    GAMEOVER
};

int main()
{
    srand(time(0));

    RenderWindow window(VideoMode(WIDTH, HEIGHT), "Car Dodging Game");
    window.setFramerateLimit(60);

    Font font;
    font.loadFromFile("assets/font/KOMIKAP_.ttf");

    Text scoreText, speedText, infoText;

    scoreText.setFont(font);
    scoreText.setCharacterSize(30);
    scoreText.setFillColor(Color::White);
    scoreText.setPosition(40, 20);

    speedText.setFont(font);
    speedText.setCharacterSize(30);
    speedText.setFillColor(Color::White);
    speedText.setPosition(550, 20);

    infoText.setFont(font);
    infoText.setCharacterSize(40);
    infoText.setFillColor(Color::Yellow);

    Music bgMusic;
    bgMusic.openFromFile("assets/audio/bg_music.ogg");
    bgMusic.setLoop(true);
    bgMusic.play();

    SoundBuffer crashBuffer;
    crashBuffer.loadFromFile("assets/audio/crash.wav");

    Sound crashSound;
    crashSound.setBuffer(crashBuffer);
```



```

PlayerCar player;
vector<EnemyCar> enemies;
vector<float> lanes = {280, 400, 520};

Clock spawnClock;

GameState gameState = MENU;

int score = 0;

float gameSpeed = 5.f, spawnInterval = 1.f;

while (window.isOpen())
{
    Event event;

    while (window.pollEvent(event))
    {
        if (event.type == Event::Closed)
            window.close();

        if (event.type == Event::KeyPressed)
        {
            if (gameState == MENU && event.key.code == Keyboard::Enter)
                gameState = PLAYING;

            else if (gameState == PLAYING && event.key.code == Keyboard::P)
                gameState = PAUSED;

            else if (gameState == PAUSED && event.key.code == Keyboard::P)
                gameState = PLAYING;

            else if (gameState == GAMEOVER && event.key.code == Keyboard::R)
            {
                enemies.clear();
                score = 0;
                gameSpeed = 5.f;
                spawnInterval = 1.f;
                bgMusic.play();
                player.getSprite().setPosition(400, 580);
                gameState = MENU;
            }
        }
    }

    if (gameState == PLAYING)
    {
        player.update(gameSpeed);

        if (spawnClock.getElapsedTime().asSeconds() > spawnInterval)
        {
            enemies.push_back(
                EnemyCar(lanes[rand() % 3], gameSpeed + (rand() % 3)));

            spawnClock.restart();
        }

        for (int i = 0; i < enemies.size(); i++)
        {
            enemies[i].update(gameSpeed);
            if (player.getBounds().intersects(enemies[i].getBounds()))
            {
                crashSound.play();
                bgMusic.stop();
                gameState = GAMEOVER;
            }
        }
    }
}

```

```

    }
}

while (!enemies.empty() && enemies.front().offScreen())
{
    enemies.erase(enemies.begin());
    score++;

    if (score % 5 == 0)
    {
        gameSpeed += 0.5f;

        if (spawnInterval > 0.4f)
            spawnInterval -= 0.05f;
    }
}

window.clear(Color(30, 30, 30));

RectangleShape road(Vector2f(400, HEIGHT));
road.setFillColor(Color::Black);
road.setPosition(200, 0);

window.draw(road);

for (int y = 0; y < HEIGHT; y += 80){
    RectangleShape line(Vector2f(12, 50));
    line.setFillColor(Color::White);
    line.setPosition(394, y);
    window.draw(line);
}

player.draw(window);

for (int i = 0; i < enemies.size(); i++)
    enemies[i].draw(window);

scoreText.setString("SCORE: " + to_string(score));
speedText.setString("SPEED: " + to_string((int)gameSpeed));

window.draw(scoreText);
window.draw(speedText);

if (gameState == MENU)
    infoText.setString("CAR DODGING GAME\n\nPress ENTER to Start");

if (gameState == PAUSED)
    infoText.setString("PAUSED\n\nPress P to Resume");

if (gameState == GAMEOVER)
    infoText.setString("GAME OVER\n\nPress R to Restart");

if (gameState != PLAYING){
    FloatRect bounds = infoText.getLocalBounds();
    infoText.setOrigin(bounds.width / 2, bounds.height / 2);

    infoText.setPosition(WIDTH / 2, HEIGHT / 2);

    window.draw(infoText);
}

window.display();
}

return 0;
}

```

Explanation :

This file is the entry point and drives the entire game loop.

It sets up the SFML window, fonts, UI text, and audio assets.

The file manages game states (menu, playing, paused, game over) and input flow.

It handles spawning enemy cars, collision checks, scoring, and difficulty scaling.

It also draws the road, lane markers, player, enemies, and status text each frame.

Car.h

```
#ifndef CAR_H
#define CAR_H

#include <SFML/Graphics.hpp>
using namespace sf;
class Car{
protected:
    Sprite sprite;
    Texture texture;

public:
    virtual void update(float speed) = 0;
    void draw(RenderWindow &window);
    FloatRect getBounds();
    Sprite &getSprite();
};
#endif
```

Explanation :

This header defines the abstract base class for all cars in the game.

It stores the common SFML sprite and texture used by derived classes.

The class enforces a virtual `update` method so each car can define its behavior.

It also declares helpers to draw the sprite and return its bounds.

These shared functions keep common rendering and collision logic in one place.

Car.cpp

```
#include "Car.h"
using namespace sf;
void Car::draw(RenderWindow &window){
    window.draw(sprite);
}

FloatRect Car::getBounds(){
    return sprite.getGlobalBounds();
}

Sprite &Car::getSprite(){
    return sprite;
}
```

Explanation :

This source file implements the common drawing and utility methods for `Car`.

`draw` renders the car sprite to the provided SFML window.

`getBounds` returns the global bounds for collision detection in the main loop.

`getSprite` exposes the sprite reference for positioning and direct access.

Keeping these in one file avoids duplication across player and enemy classes.

PlayerCar.h

```
#ifndef PLAYERCAR_H
#define PLAYERCAR_H
#include "Car.h"
class PlayerCar : public Car{
public:
    PlayerCar();
    void handleInput();
    void update(float speed) override;
};
#endif
```

Explanation :

This header declares the `PlayerCar` class that inherits from `Car`.

It exposes a constructor to load the player texture and set initial position.

It declares `handleInput` for keyboard-based lane movement.

The `update` method is overridden to run input handling each frame.

The class keeps player logic isolated from the rest of the game code.

PlayerCar.cpp

```
#include "PlayerCar.h"

using namespace sf;
namespace
{
    Texture playerTexture;
    bool loaded = false;
}

PlayerCar::PlayerCar(){
    playerTexture.loadFromFile("assets/graphics/WhiteCar.png");

    sprite.setTexture(playerTexture);
    sprite.setScale(0.4f, 0.5f);
    FloatRect bounds =sprite.getLocalBounds();

    sprite.setOrigin(bounds.width / 2.f,bounds.height / 2.f);
    sprite.setPosition(400, 580);
}

void PlayerCar::handleInput(){
    float moveSpeed = 7.f;
    if (Keyboard::isKeyPressed(Keyboard::Left)){
```

```

        if (sprite.getPosition().x > 260){
            sprite.move(-moveSpeed, 0);
        }
    }

    if (Keyboard::isKeyPressed(Keyboard::Right)){
        if (sprite.getPosition().x < 540){
            sprite.move(moveSpeed, 0);
        }
    }
}

void PlayerCar::update(float speed){
    handleInput();
}

```

Explanation :

This file loads the player car texture and configures its sprite size and origin.

It places the player car near the bottom of the road at the start of play.

`handleInput` reads arrow keys and moves the car within the road bounds.

The `'update'` method delegates to input handling, keeping movement responsive.

All player movement rules stay here for easier maintenance.

EnemyCar.h

```

#ifndef ENEMYCAR_H
#define ENEMYCAR_H

#include "Car.h"

class EnemyCar : public Car{
private:
    float enemySpeed;

public:
    EnemyCar(float x, float speed);
    void update(float speed) override;
    bool offScreen();
};

#endif

```

Explanation :

This header defines the **EnemyCar** class derived from **Car**.

It stores an **enemySpeed** value so each enemy can move independently.

The constructor accepts a lane position and starting speed for variety.

It declares **update** to move the enemy and **offScreen** to remove old cars.

These declarations keep enemy behavior distinct from the player logic

EnemyCar.cpp

```

#include "EnemyCar.h"
#include <cstdlib>
#include <iostream>

using namespace sf;

```

```

namespace{
    Texture red1;
    Texture red2;
    Texture yellow1;
    Texture yellow2;
    Texture yellow3;

    bool loaded = false;
}

EnemyCar::EnemyCar(float x, float speed){
    red1.loadFromFile("assets/graphics/RedCar1.png");
    red2.loadFromFile("assets/graphics/RedCar2.png");
    yellow1.loadFromFile("assets/graphics/YellowCar1.png");
    yellow2.loadFromFile("assets/graphics/YellowCar2.png");
    yellow3.loadFromFile("assets/graphics/YellowCar3.png");

    int type = rand() % 5;
    switch (type)
    {
        case 0:
            sprite.setTexture(red1);
            break;
        case 1:
            sprite.setTexture(red2);
            break;
        case 2:
            sprite.setTexture(yellow1);
            break;
        case 3:
            sprite.setTexture(yellow2);
            break;
        case 4:
            sprite.setTexture(yellow3);
            break;
    }
    sprite.setScale(0.4f, 0.5f);
    FloatRect bounds = sprite.getLocalBounds();

    sprite.setOrigin(
        bounds.width / 2.f,
        bounds.height / 2.f);
    sprite.setPosition(x, -150);
    enemySpeed = speed;
}

void EnemyCar::update(float speed){
    sprite.move(0, enemySpeed);
}

bool EnemyCar::offScreen(){
    return sprite.getPosition().y > 800;
}

```

Explanation :

This file loads multiple enemy car textures and randomly selects one per spawn.

It sets the enemy scale, origin, and initial off-screen position above the road.

Each enemy moves downward using its own stored speed value.

offScreen returns true when the car has moved past the bottom edge.

This logic supports random variety and smooth enemy cleanup during gameplay.